



CC1002 - Introducción a la Programación

CLASE 3 : FUNCIONES, MÓDULOS E INTERACCIÓN

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

Basado en Patricio.Inostroza@dcc.uchile.cl

Problema en parejas

Calcular la diferencia entre 2 instantes de tiempo. **Ejemplo:** entre 10:25 y 17:07 es 6:42.

Pista: una hora (10:25) se representa con 4 dígitos (**1025**).

minutos

De hora a minutos

1025 → 625

hhmm

De minutos a HHMM

625 → 1025

diferencia

Diferencia entre dos horas

1025 y 1707 → 642

¿Qué entregar? Las 3 funciones, cada una con su respectiva receta de diseño.

Solución (1/3): función minutos

```
# minutos: int -> int
# convierte una hora hhmm en los minutos transcurridos desde 00:00
# ejemplo: minutos(1025) debe producir 625

def minutos(hora):
    horas = hora // 100
    mins = hora % 100
    return horas * 60 + mins

# Test
assert minutos(1025) == 625
```

LA RECETA

Contrato, propósito, ejemplo, código y test.

Hace una sola cosa:
pasar de **hhmm** a
minutos.

Solución (2/3): función hhmm

```
# hhmm: int -> int
# convierte una cantidad de minutos en una hora con formato hhmm
# ejemplo: hhmm(625) debe producir 1025
def hhmm(totalMin):
    horas = totalMin // 60
    mins  = totalMin % 60
    return horas * 100 + mins

# Test
assert hhmm(625) == 1025
```

LA RECETA

Es la función inversa de **minutos**.

El nombre dice qué produce: una hora en formato hhmm.

Solución (3/3): función diferencia

```
# diferencia: int int -> int
# calcula el tiempo transcurrido entre dos horas hhmm (inicio <= fin)
# ejemplo: diferencia(1025, 1707) debe producir 642

def diferencia(inicio, fin):
    assert inicio <= fin
    return hhmm(minutos(fin)
                - minutos(inicio))

# Test
assert diferencia(1025, 1707) == 642
```

REUTILIZAR

No repite cálculos: usa las dos funciones anteriores.

El **assert** del cuerpo protege la restricción; el del final es la prueba.

PYTHON · CLASE 03

Variables y su alcance (scope)

...dónde vive cada variable



This Photo by Unknown Author is licensed under CC BY

Alcance de las variables

Una variable se 've' desde el momento que tiene un valor

```
>>> numero
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
NameError: name 'numero' is not defined
```

```
>>> numero = 15
```

```
⇒ 15
```



Alcance de las variables

```
>>> def sumaValores(b, c)
...     return a + b + c
>>> sumaValores(2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in suma
NameError: global name 'a' is not defined
```

El mismo problema aparece en las funciones: **a** no existe dentro de **sumaValores**.

Ojo: una variable definida fuera de la función no es visible dentro de ella.



Alcance de las variables

```
>>> def sumaValores(b , c)
...     return a + b + c
>>> sumaValores(2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in suma
NameError: global name 'a' is not defined
```

El mismo problema se presenta en las funciones

```
>>> a = 10
>>> def sumaValores(b , c)
...     return a + b + c
>>> sumaValores(2, 3)
⇒ 15
```

Si la variable tomó un valor, la función 've' el valor

Alcance de las variables

```
>>> a = 10
>>> def sumaValores(b , c)
...     return a + b + c
>>> a = 20
>>> sumaValores(2, 3)
⇒ 25
```

Si la variable tomó un valor, la función 've' el **último** valor

```
>>> def sumaValores(b , c)
...     return a + b + c
>>> a = 20
>>> sumaValores(2, 3)
⇒ 25
```

Si la variable tomó un valor, la función 've' el **último** valor



Alcance de las variables

```
>>> def sumaValores(b , c)
>>>     a = 20
...     return a + b + c
>>> sumaValores(2, 3)
⇒ 25
```

Si la variable se define dentro de la función, la función 've' ese valor

```
>>> def sumaValores(b , c)
>>>     a = 20
...     return a + b + c
>>> a = 10
>>> sumaValores(2, 3)
⇒ 25
```

Si la variable se define dentro de la función, la función 've' ese valor



Alcance de las variables

```
>>> def sumaValores(b , c)
...     return a + b + c
>>> a = 20
>>> b = 30
>>> c = 40
>>> sumaValores(1, 3)
⇒ 24
```

Los parámetros son variables dentro de la función, y la función 've' esos valores

```
>>> def sumaValores(b , c)
...     return a + b + c
>>> a = 20
>>> b = 30
>>> sumaValores(a, b)
⇒ 70
```

Los parámetros son variables dentro de la función, y la función 've' esos valores



Alcance de las variables

```
>>> def sumaValores(b , c)
>>>     a = 20
...     return a + b + c
>>> a = 10
>>> b = 30
>>> c = 40
>>> sumaValores(1, 3)
=> 24
>>> a
=> 10
>>> b
=> 30
>>> c
=> 40
```

Los parámetros y las variables de la función solo se 'ven' dentro de la función.



PYTHON · CLASE 03

Funciones, Módulos, Interacción

Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

Basado en Patricio.Inostroza@dcc.uchile.cl



Malas prácticas

Funciona... pero....



Malas prácticas

```
def areaAnillo(interior , exterior):  
    return 3.14 * exterior ** 2 - 3.14 * interior ** 2
```



- Funciona... pero
- Difícil de mantener
 - Todo junto se hace difícil de mantener
- **Divide y Reinarás**
 - Dividir en tareas simples
 - Código que se entiende
 - Ayuda a acotar rápidamente los problemas

```
# Buena práctica  
def areaAnillo(interior , exterior):  
    return areaCirculo(exterior) - areaCirculo(interior)
```



PYTHON · CLASE 03

Descomposición funcional

Dividir para reinar

```
def leer_datos()
```

```
def procesar()
```

```
def mostrar()
```

Problema: Funciones simples

Una importante cadena de cines de Santiago tiene completa libertad para fijar los precios de las entradas. Claramente, mientras más cara sea la entrada, menos personas estarán dispuestas a pagar por ellas. En un reciente estudio de mercado, se determinó que hay una relación entre el precio al que se venden las entradas y la cantidad de espectadores promedio: a un precio de 5.000 por entrada, 120 personas van a ver la película; al reducir en 500 el precio de la entrada, los espectadores aumentan en 15. Desafortunadamente, mientras más personas ocupan la sala para ver la película, más se debe gastar en limpieza y mantenimiento general. Para reproducir una película, el cine gasta 180.000. Asimismo, se gastan en promedio 40 por espectador por conceptos de limpieza y mantenimiento. El gerente del cine le encarga determinar cuál es la relación exacta entre las ganancias y el precio de las entradas para poder decidir a qué precio se debe vender cada entrada para maximizar las ganancias totales.

Problema: Funciones simples

Una importante cadena de cines de Santiago tiene completa libertad para fijar los precios de las entradas. Claramente, mientras más cara sea la entrada, menos personas estarán dispuestas a pagar por ellas. En un reciente estudio de mercado, se determinó que hay una relación entre el precio al que se venden las entradas y la cantidad de espectadores promedio: a un precio de 5.000 por entrada, 120 personas van a ver la película; al reducir en 500 el precio de la entrada, los espectadores aumentan en 15. Desafortunadamente, mientras más personas ocupan la sala para ver la película, más se debe gastar en limpieza y mantenimiento general. Para reproducir una película, el cine gasta 180.000. Asimismo, se gastan en promedio 40 por espectador por conceptos de limpieza y mantenimiento. El gerente del cine le encarga determinar cuál es la relación exacta entre las ganancias y el precio de las entradas para poder decidir a qué precio se debe vender cada entrada para maximizar las ganancias totales.

Ingresos: valor de la entrada por el N^{ro.} de espectadores

Gastos: gasto fijo + gasto variable

gasto fijo: 180.000 pesos

gasto variable: 40 pesos por espectador

Ganancias: ingresos - gastos

¿Y los espectadores?

Espectadores: Fijo + variables

Fijos: 120 espectadores si el precio es \$5.000

Variable: +/-15 espectadores si el precio baja/sube \$500



Problema: Funciones simples

Espectadores: Fijo + variables

Fijos: 120 espectadores si el precio es \$5.000

Variable: +/-15 espectadores si el precio baja/sube \$500

- Si el precio es 5000 $\rightarrow$ 120 espectadores
- Si el precio sube en 500 pesos, hay 15 espectadores menos
- Si el precio baja en 500 pesos, hay 15 espectadores más
- Diferencia = $5000 - \text{precio}$
- factor = $\text{Diferencia} // 500$
- nuevosEspectadores = $\text{factor} * 15$
- Espectadores = $120 + \text{nuevosEspectadores}$
- Espectadores = $120 + (5000 - \text{precio}) // 500 * 15$

Problema: Funciones simples

```
# espectadores: int -> int
# calcular el numero de espectadores, dado precioEntrada3
# ejemplo: espectadores(5000) retorna 120; espectadores(5500) retorna 105
def espectadores(precioEntrada):
    return 120 + (5000 - precioEntrada) // 500 * 15
# Test
assert espectadores(5500) == 105
```

Espectadores: Fijo + variables

Fijos: 120 espectadores si el precio es \$5.000

Variable: +/-15 espectadores si el precio baja/sube \$500



Problema: Funciones simples

```
# ingreso: int -> int
# calcular el ingreso total, dado precioEntrada
# ejemplo: ingreso (5000) debe retornar 600.000
def ingreso(precioEntrada):
    return precioEntrada * espectadores(precioEntrada)
# Test
assert ingreso(5000) == (5000 * 120)
```

Ingresos: valor de la entrada por el N^{ro.} de espectadores



Problema: Funciones simples

```
# gasto: int -> int
# calcular el gasto total, dado precioEntrada
# ejemplo: gasto(5000) es 184800
def gasto(precioEntrada):
    return 180000 + 40 * espectadores(precioEntrada)
# Test
assert gasto(5000) == (180000 + 40 * 120 )
```

Gastos: gasto fijo + gasto variable
; gasto fijo: 180.000
gasto variable: 40 por espectador

Problema: Funciones simples

```
# ganancia: int -> int
# calcular ganancia, dado precioEntrada
# ejemplo: ganancia(5000) es 600.000 - 184800 = 415200
def ganancia(precioEntrada):
    return ingresos(precioEntrada) - gasto(precioEntrada)
# test
assert ganancia(5000) == 415200
```

Ganancias: ingresos - gastos



Problema: Funciones simples

```
# espectadores: int -> int
# calcular el numero de espectadores, dado precioEntrada
def espectadores(precioEntrada):
    return 120 + (5000 - precioEntrada) * 15 // 500
# Test
assert espectadores(5500) == 105
```

```
# ingreso: int -> int
# calcular el ingreso total, dado precioEntrada
def ingreso(precioEntrada):
    return precioEntrada * espectadores(precioEntrada)
# Test
assert ingreso(5000) == (5000 * 120)
```

```
# gasto: int -> int
# calcular el gasto total, dado precioEntrada
def gasto(precioEntrada):
    return 180000 + 40 * espectadores(precioEntrada)
# Test
assert gasto(5000) == (180000 + 40 * 120 )
```

```
# ganancia: int -> int
# calcular ganancia, dado precioEntrada
def ganancia(precioEntrada):
    return ingresos(precioEntrada) - gasto(precioEntrada)
```



PYTHON · CLASE 03

Interacción

...conversando con la persona usuaria

```
input()
```

```
print()
```

```
int()
```

Interacción

```
>>># ingreso de string
>>> nombre = input('dime tu nombre? ')
           ⇒ dime un nombre ? juan
>>> nombre
           ⇒ 'juan'
```

La función **input** siempre retorna un String

```
>>># ingreso de un número
>>> edad = int( input('dime tu edad? '))
           ⇒ dime tu edad? 18
>>> edad
           ⇒ 18
```

Para leer un entero se debe hacer un cast



Interacción

```
>> print("hola")
=> hola

>> print("hola Juan")
=> hola Juan

>> print("hola" + "Juan")
=> holaJuan

>> print("hola" , "Juan")
=> hola Juan

>> print("hola□" + "Juan"). # □ : Solo para indicar que hay un espacio
=> hola Juan

>> print("hola□" + 'Juan')
=> hola Juan

>> print("hola□" + '"Juan"')
=> hola "Juan"
```



Interacción

```
>> print("hola")
=> hola

>> print("hola Juan")
=> hola Juan

>> print("hola" + "Juan")
=> holaJuan

>> print("hola" , "Juan")
=> hola Juan

>> print("hola_" + "Juan")
=> hola Juan

>> print("hola_" + 'Juan')
=> hola Juan

>> print("hola_" + '"Juan"')
=> hola "Juan"

>> print("'hola_' + '"Juan"'')
=> 'hola ' "Juan"
```

```
>> print(100)
```

```
=> 100
```

```
>> print(100 + 1)
```

```
=> 101
```

```
>> print("La edad es " + 100)
```

Traceback (most recent call last):

File "<string>", line 12, in <module>

TypeError: can only concatenate str (not "int") to str



PYTHON · CLASE 03

Módulos

...organizando los archivos .py

main.py

utils.py

juego.py

Módulos y Archivos .py

- El código se puede guardar
 - En archivos de texto con extensión **.py**
- **Módulo**
 - Archivo que agrupa funciones de un mismo tema
 - Archivo de Python
- Otro archivo puede "cargar" el módulo
 - **import** archivo
 - **from** archivo **import** funcion1, funcion2, ...
 - **from** archivo **import** *



Problema del Cine como Módulo

cine.py

```
# espectadores: int -> int
# calcular el numero de espectadores, dado precioEntrada
# ejemplo: espectadores(5000) retorna 120; espectadores(5500) retorna 105
def espectadores(precioEntrada):
    return 120 + (5000 - precioEntrada) / 500 * 500
# Test
assert espectadores(5500) == 105

# ingreso: int -> int
# calcular el ingreso total, dado precioEntrada
# ejemplo: ingreso (5000) debe retornar 600.000
def ingreso(precioEntrada):
    return precioEntrada * espectadores(precioEntrada)
# Test
assert ingreso(5000) == (5000 * 120)

# gasto: int -> int
# calcular el gasto total, dado precioEntrada
# ejemplo: gasto(5000) es 184800
def gasto(precioEntrada):
    return 180000 + 40 * espectadores(precioEntrada)
# Test
assert gasto(5000) == (180000 + 40 * 120)

# ganancia: int -> int
# calcular ganacia, dado precioEntrada
# ejemplo: ganancia(5000) es 600.000 - 184800 = 415200
def ganancia(precioEntrada):
    return ingresos(precioEntrada) - gasto(precioEntrada)
# test
assert ganancia(5000) == 415200
```

tarea.py

```
import cine

precio = 5500
publico = cine.espectadores(precio)
margen = cine.ganancia(precio)

print("Precio entrada: ", precio)
print("Nro. Espectadores: ", publico)
print("La ganancia es: ", margen)
```



Problema del Cine como Módulo

cine.py

```
# espectadores: int -> int
# calcular el numero de espectadores, dado precioEntrada
# ejemplo: espectadores(5000) retorna 120; espectadores(5500) retorna 105
def espectadores(precioEntrada):
    return 120 + (5000 - precioEntrada) / 500 * 500
# Test
assert espectadores(5500) == 105

# ingreso: int -> int
# calcular el ingreso total, dado precioEntrada
# ejemplo: ingreso (5000) debe retornar 600.000
def ingreso(precioEntrada):
    return precioEntrada * espectadores(precioEntrada)
# Test
assert ingreso(5000) == (5000 * 120)

# gasto: int -> int
# calcular el gasto total, dado precioEntrada
# ejemplo: gasto(5000) es 184800
def gasto(precioEntrada):
    return 180000 + 40 * espectadores(precioEntrada)
# Test
assert gasto(5000) == (180000 + 40 * 120)

# ganancia: int -> int
# calcular ganancia, dado precioEntrada
# ejemplo: ganancia(5000) es 600.000 - 184800 = 415200
def ganancia(precioEntrada):
    return ingresos(precioEntrada) - gasto(precioEntrada)
# test
assert ganancia(5000) == 415200
```

tarea.py

```
from cine import espectadores, ganancia

precio = 5500
publico= espectadores(precio )
margen= ganancia(precio )

print("Precio entrada: ", precio)
print("Nro. Espectadores: ", publico)
print("La ganancia es: ", margen)
```



Problema del Cine como Módulo

cine.py

```
# espectadores: int -> int
# calcular el numero de espectadores, dado precioEntrada
# ejemplo: espectadores(5000) retorna 120; espectadores(5500) retorna 105
def espectadores(precioEntrada):
    return 120 + (5000 - precioEntrada) / 500 * 500
# Test
assert espectadores(5500) == 105

# ingreso: int -> int
# calcular el ingreso total, dado precioEntrada
# ejemplo: ingreso (5000) debe retornar 600.000
def ingreso(precioEntrada):
    return precioEntrada * espectadores(precioEntrada)
# Test
assert ingreso(5000) == (5000 * 120)

# gasto: int -> int
# calcular el gasto total, dado precioEntrada
# ejemplo: gasto(5000) es 184800
def gasto(precioEntrada):
    return 180000 + 40 * espectadores(precioEntrada)
# Test
assert gasto(5000) == (180000 + 40 * 120)

# ganancia: int -> int
# calcular ganacia, dado precioEntrada
# ejemplo: ganancia(5000) es 600.000 - 184800 = 415200
def ganancia(precioEntrada):
    return ingresos(precioEntrada) - gasto(precioEntrada)
# test
assert ganancia(5000) == 415200
```

tarea.py

from cine import *

```
precio = 5500
publico= espectadores(precio )
margen= ganancia(precio )

print("Precio entrada: ", precio)
print("Nro. Espectadores: ", publico)
print("La ganancia es: ", margen)
```



Módulo

- `import cine`
 - Importa todas las funciones del módulo
 - Requiere del nombre del módulo para usar una función

```
>>> print(cine.espectadores(5500))
```

⇒ 105
- `from cine import espectadores`
 - Importa solo la función `espectadores`
 - Ventaja: no requiere el nombre del módulo

```
>>> print(espectadores(5500))
```

⇒ 105
- `from cine import *`
 - Importa **todas** las funciones del módulo



- En el restorán "Kdo K'n Apeto", el costo de hacer un menú se compone del sueldo de los cocineros, los insumos y el aseo. El costo de un día de trabajo de los cocineros es 100 mil pesos. El costo promedio de preparar un plato es de 1.500 pesos y se vende a 3.500 pesos. A ese precio de venta ingresan 35 comensales en 1 hora. El costo por aseo de cada comensal es de 100 pesos. Si el precio del plato sube 250 pesos, el número de comensales baja en 5 por hora. Si el precio baja en 250 pesos, ingresan 5 comensales más por hora.
- Sabiendo que el precio de venta de los platos del día se fija en la mañana, programe las funciones que permitan obtener los costos e ingresos por hora. También programe la función que retorne las ganancias (o pérdidas) que se tendrán luego de **ocho** horas de trabajo.



- En el restorán "Kdo K'n Apeto", el costo de hacer un menú se compone del sueldo de los cocineros, los insumos y el aseo. El costo de un día de trabajo de los cocineros es 100 mil pesos. El costo promedio de preparar un plato es de 1.500 pesos y se vende a 3.500 pesos. A ese precio de venta ingresan 35 comensales en 1 hora. El costo por aseo de cada comensal es de 100 pesos. Si el precio del plato sube 250 pesos, el número de comensales baja en 5 por hora. Si el precio baja en 250 pesos, ingresan 5 comensales más por hora.

- Sabiendo que el precio de venta de los platos del día se fija en la mañana, programe las funciones que permitan obtener los costos e ingresos por hora. También programe la función que retorne las ganancias (o pérdidas) que se tendrán luego de ocho horas de trabajo.



Costo Fijo x Dia: 100 mil pesos

Costo Variable por hora: $(1500 + 100) * \text{NroComensalesXhora}$

Ingresos X Hora: $3.500 * \text{NroComensalesXhora}$

$\text{NroComensalesXHora} = 35 + (3500 - \text{precio}) / 250 * 5$

Ganancia x Dia: $(\text{IngresosXhora} - \text{CostoXHora}) * 8$

Ejercicio propuesto

- Implementar las funciones del restorán Kdo K'n Apeto en un archivo de nombre `kdoknapeto.py`
- Cree un nuevo archivo que cargue las funciones de `kdoknapeto`
- Cree un diálogo similar a:
 - Estimador de ingresos del restorán Kdo K'n Apeto
 - ¿Cuál será hoy el precio del menú? 4000
 - ¿Cuántas horas trabajarán hoy? 8
 - Ingresarán XXX comensales en 1 hora
 - Las 8 horas de trabajo dejarían YYY pesos de margen

